

Salesforce Developer Bridge Program

Day 1 – Self Learning Track Report

Hospital OPD Management System

Student Details

Name: Kuppili Sudharshan

Branch: CSE

Register Number: 24PA5A0517

Program: Salesforce Developer Bridge Program

Scenario Chosen: Hospital OPD Management System

Task 1 – Data Modeling & Object Relationships

Objective

To understand Salesforce Data Modeling by creating custom objects, custom fields, and setting up relationships between them.

Scenario

Hospital OPD Management System

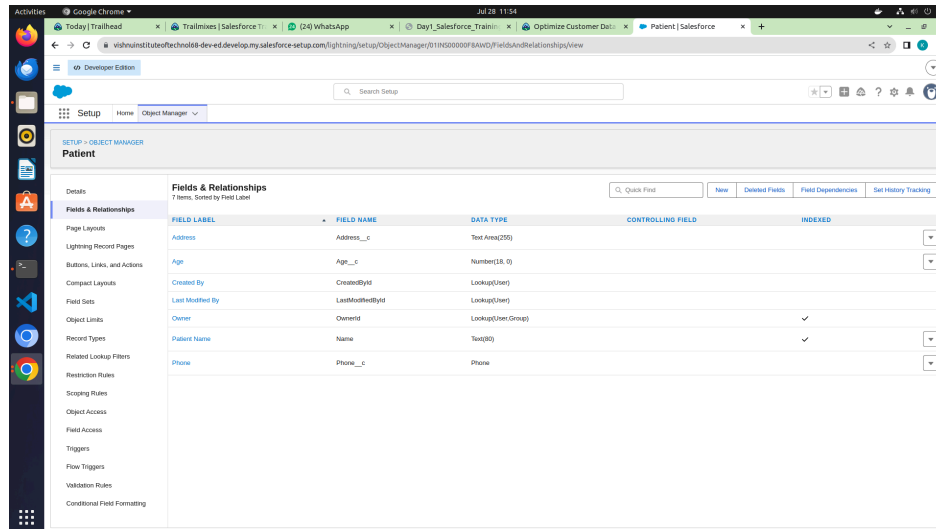
Steps Performed

- Created two custom objects: Patient__c and Appointment__c.
- Added the required fields to each object.
- Connected the two objects using a Lookup Relationship.

Objects / Fields Created

Patient__c

- Name
- Age (Number)
- Phone (Phone)
- Address (Text Area)



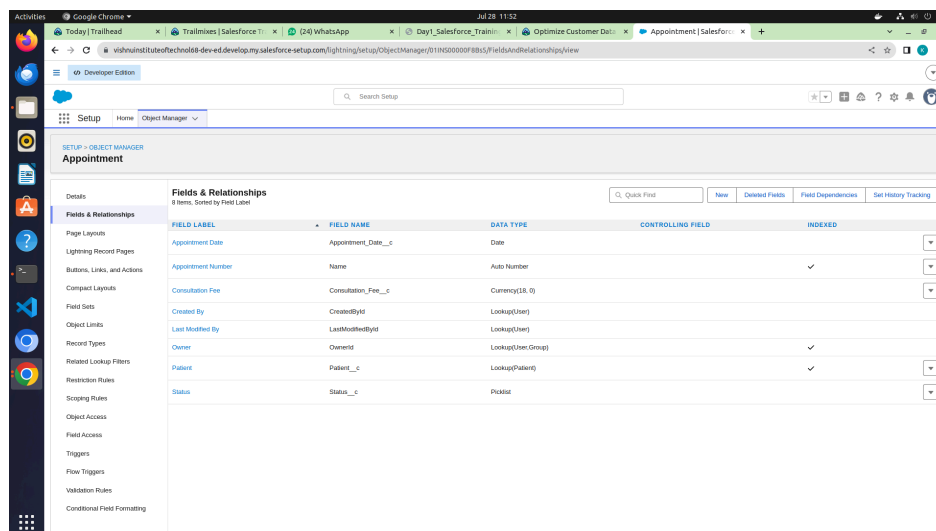
The screenshot shows the Salesforce Setup interface for the 'Patient' object. The 'Fields & Relationships' tab is selected, displaying a table of 7 fields. The fields are: Address (Text Area(255)), Age__c (Number(18, 0)), Created By (Lookup(User)), Last Modified By (Lookup(User)), Owner (Lookup(User Group)), Patient Name (Text(80)), and Phone (Phone). The 'Patient Name' and 'Phone' fields are marked as indexed.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Address	Address__c	Text Area(255)		
Age	Age__c	Number(18, 0)		
Created By	CreatedBy	Lookup(User)		
Last Modified By	LastModifiedBy	Lookup(User)		
Owner	OwnerId	Lookup(User Group)		
Patient Name	Name	Text(80)		✓
Phone	Phone__c	Phone		✓

Figure 1: Patient Object – Fields & Relationships

Appointment__c

- Appointment Number (Auto Number)
- Appointment Date (Date)
- Status (Picklist)
- Consultation Fee (Currency)
- Patient (Lookup Relationship)



The screenshot shows the Salesforce Setup interface for the 'Appointment' object. The 'Fields & Relationships' tab is selected, displaying a table of 8 fields. The fields are: Appointment Date (Date), Appointment Number (Auto Number), Consultation Fee (Currency(18, 0)), Created By (Lookup(User)), Last Modified By (Lookup(User)), Owner (Lookup(User Group)), Patient (Lookup(Patient)), and Status (Picklist). The 'Appointment Number', 'Patient', and 'Status' fields are marked as indexed.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Appointment Date	Appointment_Date__c	Date		
Appointment Number	Name	Auto Number		✓
Consultation Fee	Consultation_Fee__c	Currency(18, 0)		
Created By	CreatedBy	Lookup(User)		
Last Modified By	LastModifiedBy	Lookup(User)		
Owner	OwnerId	Lookup(User Group)		
Patient	Patient__c	Lookup(Patient)		✓
Status	Status__c	Picklist		✓

Figure 2: Appointment Object – Fields & Relationships (Patient Lookup field visible)

Relationship

A Lookup Relationship was created between Appointment__c and Patient__c (see the Patient lookup field in the screenshot above). This lets one patient have multiple appointments, while every appointment stays linked to the right patient.

Result

The Hospital OPD data model was built successfully with the required custom objects, fields, and relationship.

Learning Outcome

Learned how to design a simple data model in Salesforce and how Lookup relationships connect two objects together.

Task 2 – Apex Basics

Objective

To understand the basics of Apex programming and debugging using the Developer Console.

Code

```
Integer age = 20;  
String patientName = 'Rahul';  
  
System.debug(age);  
System.debug(patientName);
```

Explanation

This is a simple Apex program that stores a patient's age and name in variables and prints them using System.debug(), so the values can be checked in the Debug Logs.

Result

The program ran without errors, and both values were shown correctly in the Debug Logs.

Learning Outcome

Learned the basic syntax of Apex, how to declare variables, and how to check output using the Developer Console.

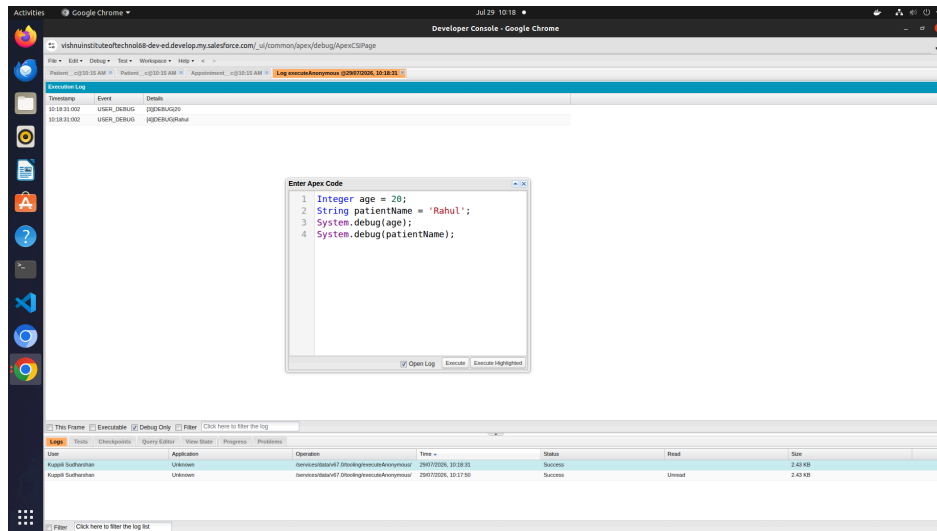


Figure 3: Execute Anonymous Window with Apex Code and Debug Log Output (age = 20, patientName = Rahul)

Task 3 – SOQL Queries

Objective

To retrieve data from Salesforce custom objects using SOQL, covering a basic select, a WHERE/comparison filter, sorting with LIMIT, a parent-child relationship query, and an aggregate query.

SOQL Queries

Query 1 – Basic SELECT

```
SELECT Name
FROM Patient__c
```

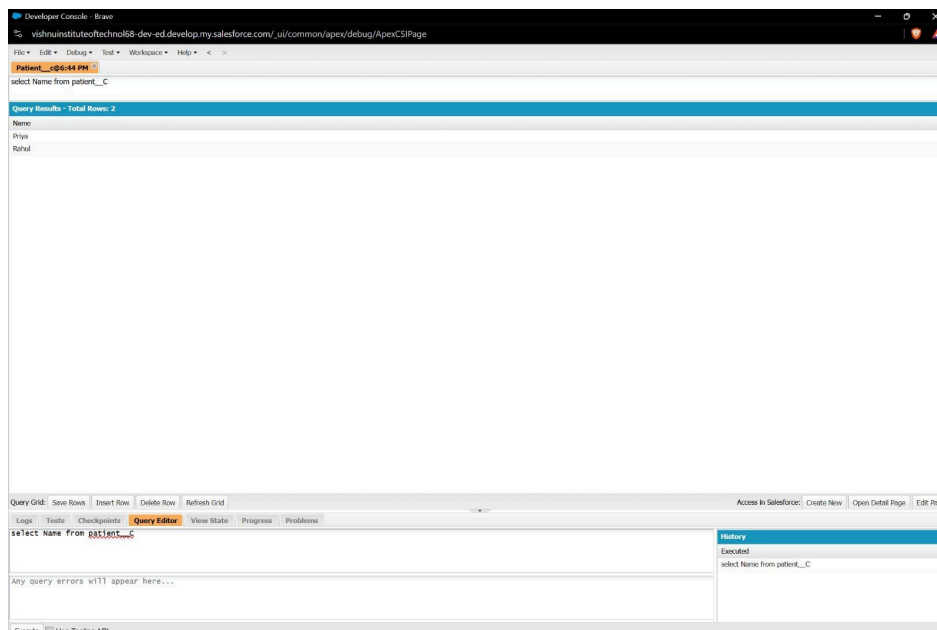
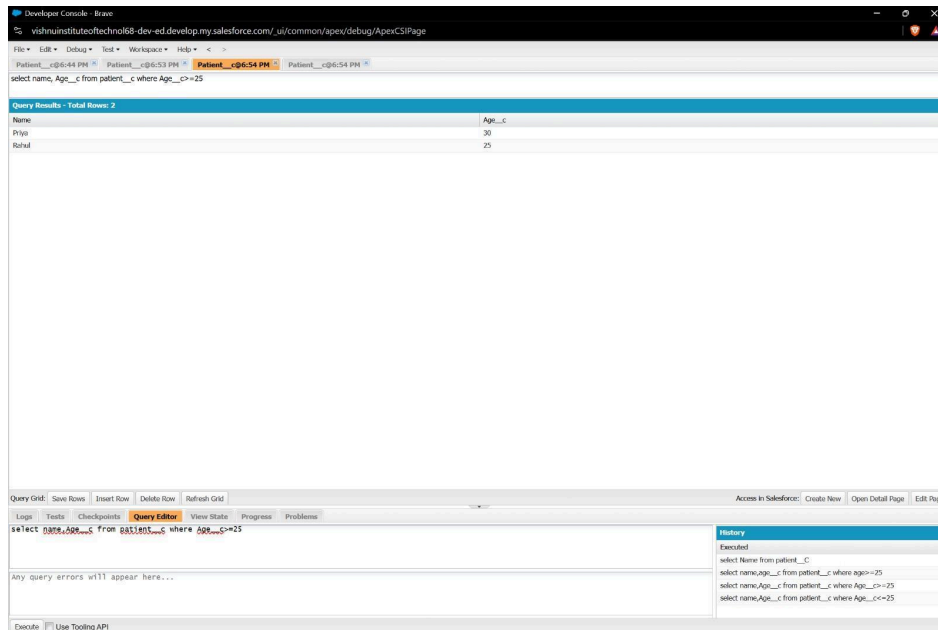


Figure 4: SOQL Query 1 Output – Basic Select on Patient__c

Query 2 – WHERE Clause with Comparison Operator

```
SELECT Name, Age__c
FROM Patient__c
WHERE Age__c >= 25
```



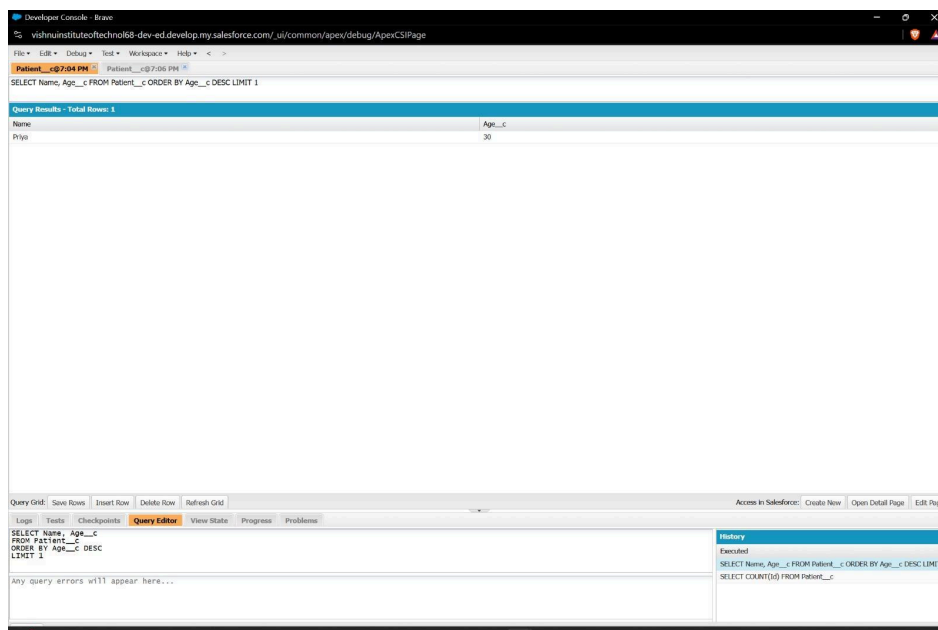
The screenshot shows the Salesforce Developer Console with a SOQL query executed. The query is: `select name, Age__c from patient__c where Age__c >= 25`. The results table shows two rows: one for 'Priya' with age 30, and one for 'Rahul' with age 25. The console interface includes tabs for 'Query Editor', 'View State', 'Progress', and 'Problems'. A 'History' panel on the right shows the executed query.

Name	Age__c
Priya	30
Rahul	25

Figure 5: SOQL Query 2 Output – Patients with Age >= 25

Query 3 – ORDER BY with LIMIT

```
SELECT Name, Age__c
FROM Patient__c
ORDER BY Age__c DESC
LIMIT 1
```



The screenshot shows the Salesforce Developer Console with a SOQL query executed. The query is: `SELECT Name, Age__c FROM Patient__c ORDER BY Age__c DESC LIMIT 1`. The results table shows one row: 'Priya' with age 30. The console interface includes tabs for 'Query Editor', 'View State', 'Progress', and 'Problems'. A 'History' panel on the right shows the executed query.

Name	Age__c
Priya	30

Figure 6: SOQL Query 3 Output – Oldest Patient (Sorted, Limited to 1)

Query 4 – Relationship Query (Child-to-Parent)

```
SELECT Appointment_Date__c, Status__c, Patient__r.Name  
FROM Appointment__c
```

This query pulls the appointment date and status along with the related patient's name, using the child-to-parent relationship path Patient__r.Name.

Query 5 – Aggregate Query with COUNT()

```
SELECT COUNT(Id)  
FROM Patient__c
```

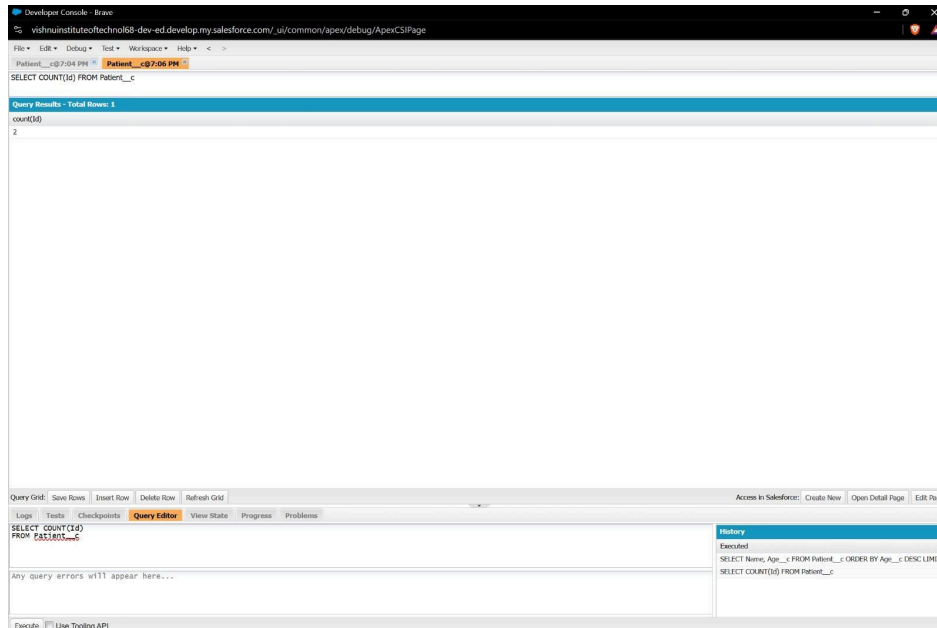


Figure 7: SOQL Query 5 Output – Total Patient Count

Result

All five SOQL queries ran successfully in the Developer Console and returned the expected records from the Hospital OPD Management System.

Learning Outcome

Learned how to write SOQL queries using WHERE filters, comparison operators, ORDER BY with LIMIT, relationship queries across parent and child objects, and aggregate functions like COUNT().

Task 4 – Apex Trigger

Objective

To prevent duplicate appointments for the same patient on the same date, using an Apex Trigger and a Handler Class.

Trigger

```

trigger AppointmentTrigger on Appointment__c (before insert) {

    AppointmentTriggerHandler.preventDuplicateAppointments(Trigger.new);

}

```

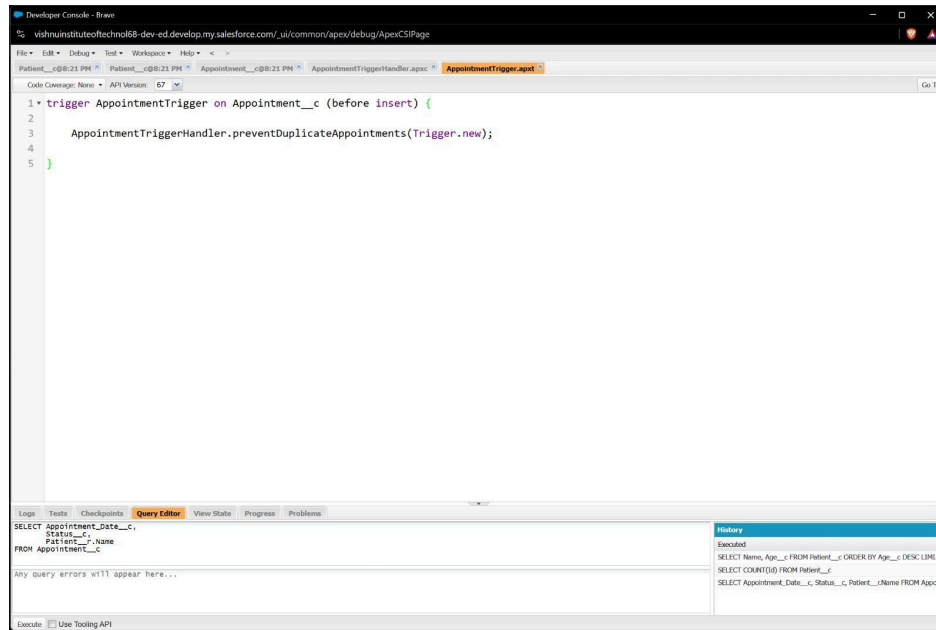


Figure 8: Appointment Trigger Code

Handler Class

```

public class AppointmentTriggerHandler {

    public static void preventDuplicateAppointments(List<Appointment__c>
newAppointments){

        Set<Id> patientIds = new Set<Id>();

        for(Appointment__c app : newAppointments){
            if(app.Patient__c != null){
                patientIds.add(app.Patient__c);
            }
        }

        List<Appointment__c> existingAppointments = [
            SELECT Id, Patient__c, Appointment_Date__c
            FROM Appointment__c
            WHERE Patient__c IN :patientIds
        ];

        for(Appointment__c newApp : newAppointments){

            for(Appointment__c oldApp : existingAppointments){

                if(newApp.Patient__c == oldApp.Patient__c &&
                    newApp.Appointment_Date__c == oldApp.Appointment_Date__c){

```

```

        newApp.addError('Duplicate appointment is not allowed.');
```

```

    }

}

}

}

}

```

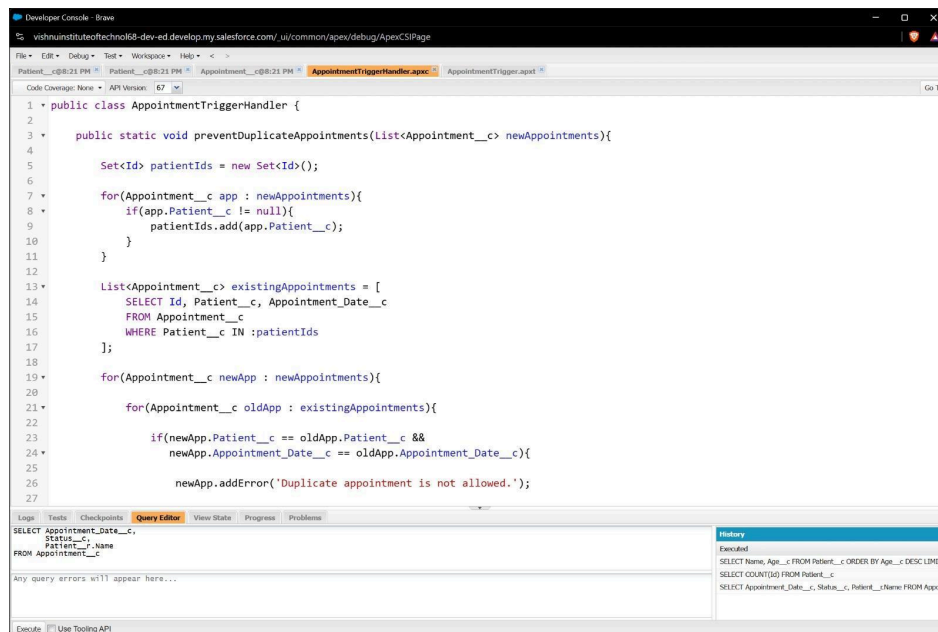


Figure 9: Trigger Handler Class Code

Explanation

The trigger fires before a new Appointment__c record is inserted and calls the handler class. The handler class checks all existing appointments for the same patient and blocks the insert if an appointment already exists for the same date, using addError().

Result

The trigger works as expected. When a duplicate appointment is entered for the same patient on the same date, Salesforce blocks it and shows the message "Duplicate appointment is not allowed." This stops duplicate appointment records from being saved.

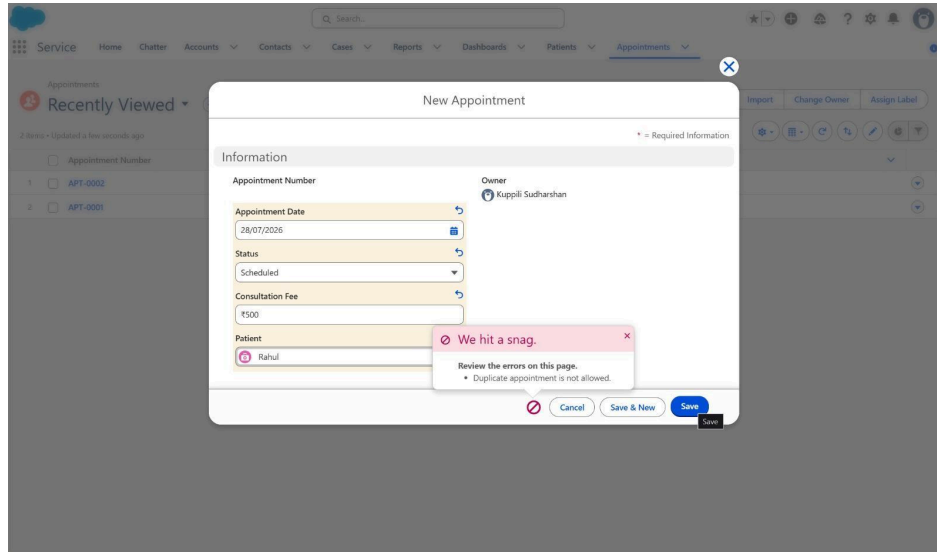


Figure 10: Duplicate Appointment Error Message

Learning Outcome

Learned how Apex triggers work, why logic should be kept in a separate handler class instead of the trigger body, and how to stop bad data from being saved using `addError()`.

Task 5 – Lightning Web Component (LWC)

Objective

To create and deploy a basic Lightning Web Component using Visual Studio Code and Salesforce CLI, and display it on a Lightning App Page.

Development Steps

- Installed Salesforce CLI.
- Created a Salesforce DX Project named HospitalOPD.
- Connected my Trailhead Playground using Salesforce CLI.
- Generated a Lightning Web Component named patientGreeting.
- Updated the HTML, JavaScript, and metadata files.
- Deployed the component to Salesforce.
- Created a Lightning App Page.
- Added the patientGreeting component to the page.
- Activated the page successfully.

Screenshots

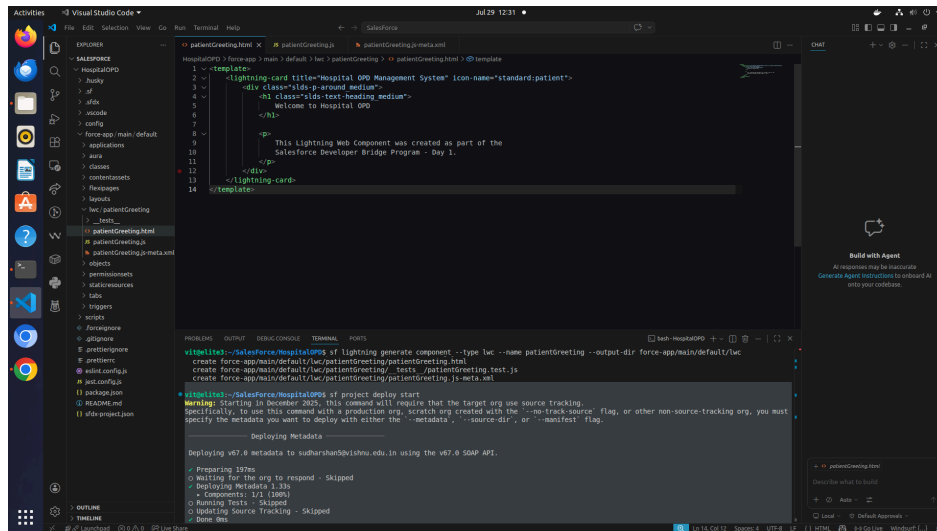


Figure 11: Salesforce DX Project (HospitalOPD) Opened in VS Code

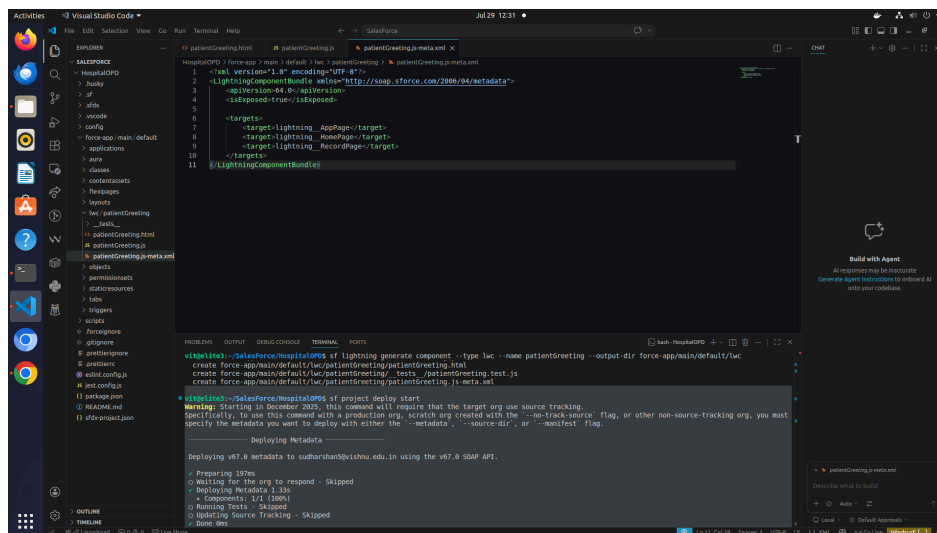


Figure 12: Successful Salesforce CLI Deployment (sf project deploy start)

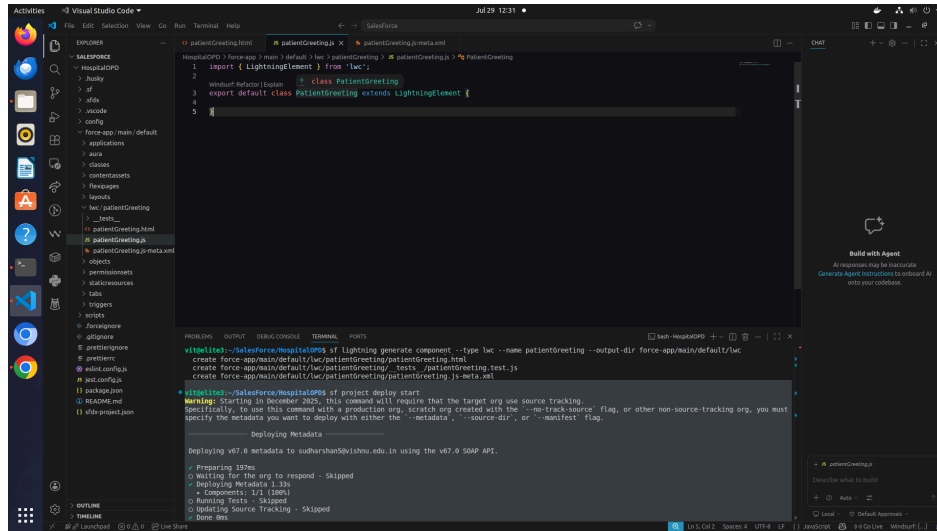


Figure 13: patientGreeting Component Files (html, js, meta.xml)

Component Description

The component shows a Lightning Card with the title "Hospital OPD Management System" and a short welcome message for users. It shows that the Lightning Web Component was created and deployed successfully.

Files Created

- patientGreeting.html
- patientGreeting.js
- patientGreeting.js-meta.xml

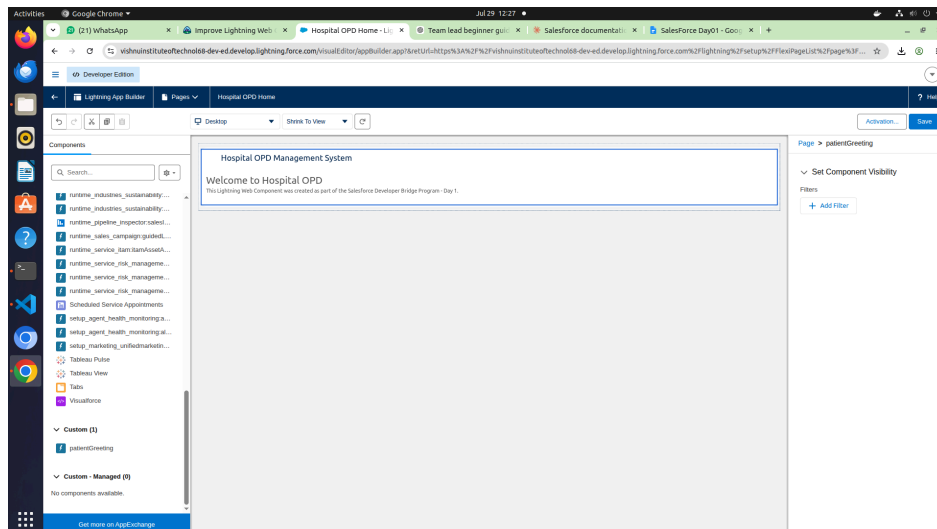


Figure 14: Lightning App Builder Showing the patientGreeting Component

Result

The patientGreeting component was added to the Hospital OPD Home page in Lightning App Builder and shows the Hospital OPD Management System card with the welcome message, exactly as designed. The page was activated successfully.

Learning Outcome

- Learned how to set up Salesforce CLI and VS Code.
 - Understood the structure of a Lightning Web Component.
 - Learned how to deploy metadata to Salesforce.
 - Learned how to use Lightning App Builder.
 - Successfully created, deployed, and activated an LWC.
-

Learning Outcomes

During Day 1 of the Salesforce Developer Bridge Program, I learned:

- Salesforce Data Modeling
- Custom Objects
- Custom Fields
- Lookup Relationships
- Apex Programming Basics
- Apex Classes
- SOQL Queries (filters, sorting, relationships, aggregates)
- Apex Triggers
- Trigger Handler Classes
- Salesforce Developer Console
- Debug Logs
- Lightning Web Components (LWC)
- Salesforce CLI and VS Code Setup
- Lightning App Builder